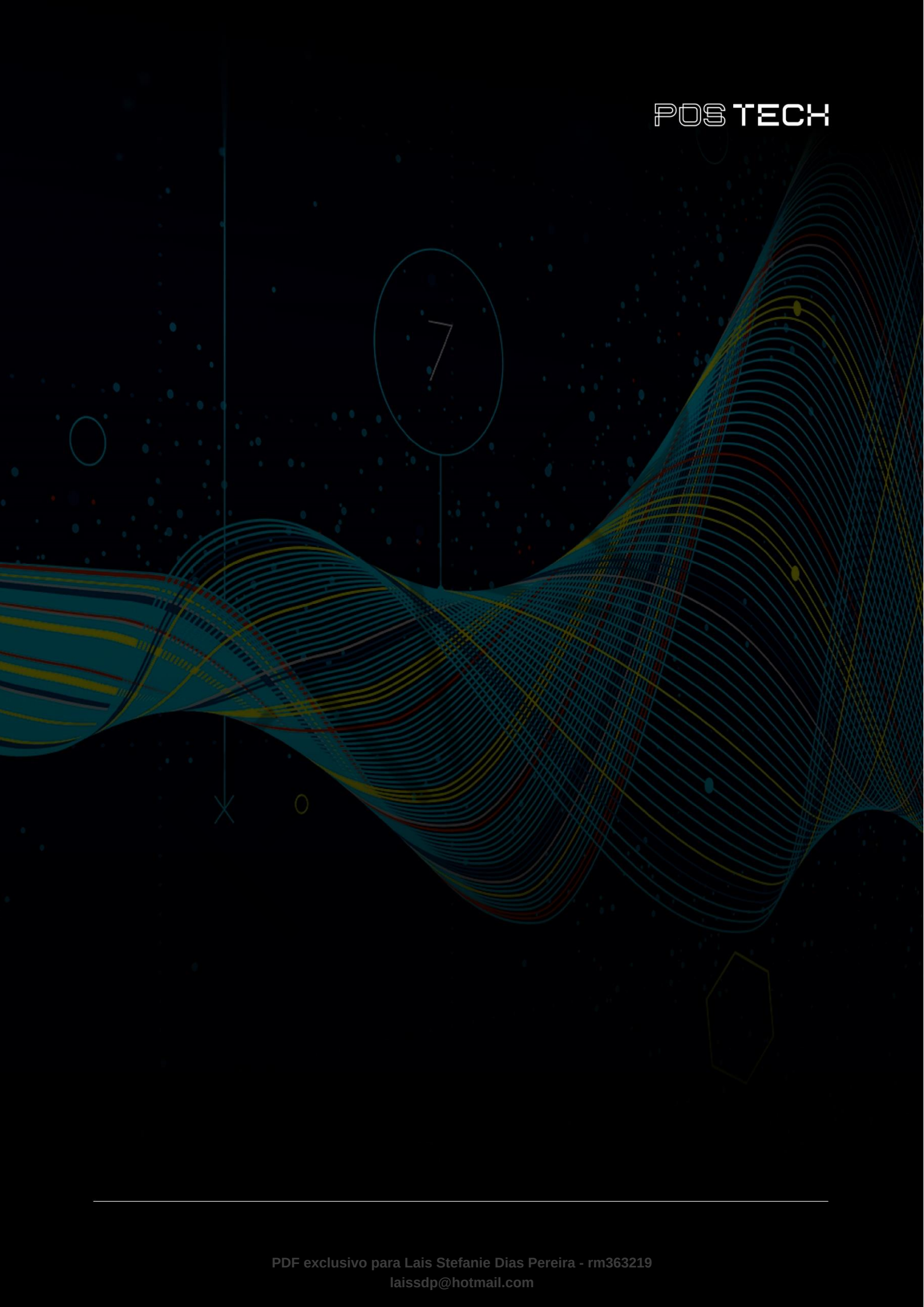




## SUMÁRIO

|                                   |    |
|-----------------------------------|----|
| O QUE VEM POR AÍ? .....           | 3  |
| HANDS ON .....                    | 4  |
| SAIBA MAIS .....                  | 5  |
| MERCADO, CASES E TENDÊNCIAS ..... | 42 |
| O QUE VOCÊ VIU NESTA AULA? .....  | 43 |
| REFERÊNCIAS.....                  | 44 |

## O QUE VEM POR AÍ?

Nesta aula, trabalharemos com Flask para criar APIs REST, que é uma ferramenta fundamental para integrar sistemas e construir aplicações modernas. Vamos explorar conceitos básicos de APIs, o que é REST, como o Flask facilita o desenvolvimento e as práticas para validação e estruturação de dados.



## HANDS ON

A seguir, serão abordados os conceitos fundamentais de APIs (Interface de Programação de Aplicações), com foco no protocolo HTTP e na arquitetura REST.

Será explicado como o HTTP funciona, incluindo métodos como GET, POST, PUT, DELETE e HEAD, além de códigos de status comuns, como 200, 404 e 500. A aula também introduzirá o Flask, uma biblioteca Python para desenvolvimento de APIs REST, com exemplos práticos de como criar endpoints e implementar métodos HTTP.

Além disso, será explorada a validação de dados em APIs utilizando a biblioteca Pydantic, que facilita a garantia de integridade e segurança dos dados recebidos. Ao final, os alunos e alunas terão a oportunidade de aplicar esses conceitos em exercícios práticos, criando e testando APIs simples com Flask e Pydantic.

## SAIBA MAIS

### API

API (Interface de Programação de Aplicações) é um conjunto de regras que permite que diferentes sistemas se comuniquem. Nesta aula, serão apresentados os conceitos do protocolo HTTP e arquitetura REST, que são fundamentais para o desenvolvimento e consumo de APIs modernas.

O protocolo HTTP (ou Hypertext Transfer Protocol), é a base de comunicação da web, permitindo a transferência de dados entre clientes e servidores. Já a arquitetura REST (Representational State Transfer) é um tipo de arquitetura que utiliza o HTTP como protocolo de comunicação.

Combinando o HTTP e a arquitetura REST, as APIs se tornam mais consistentes, fáceis de usar e podem ser mantidas com maior flexibilidade, sendo amplamente adotadas em aplicações web e integrações entre sistemas.

### Protocolo HTTP

O HTTP é um protocolo de comunicação utilizado para transferir dados na web. Ele é a base da comunicação entre navegadores (clientes) e servidores na Internet. Sempre que você acessa um site, seu navegador utiliza HTTP para enviar uma requisição ao servidor e receber a resposta.

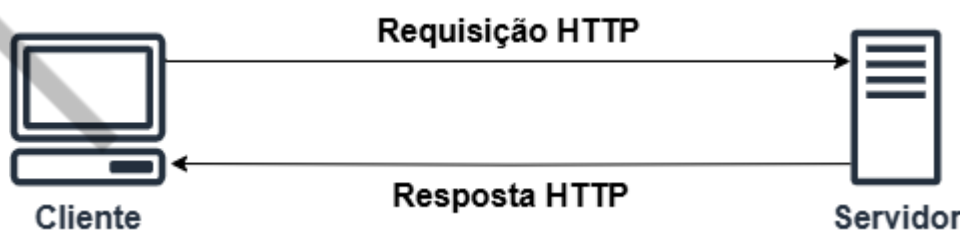


Figura 1 - Cliente web e servidor  
Fonte: Elaborado pelo autor (2025)

Observando a Figura 1, podemos entender como a arquitetura HTTP funciona:

**Cliente faz uma requisição HTTP:**

- Por exemplo, quando você digita uma URL no navegador, ele faz uma solicitação HTTP ao servidor correspondente;
- A requisição especifica o tipo de ação que o cliente quer realizar (ex.: obter um recurso, enviar dados etc.).

**Servidor responde:**

- O servidor processa a solicitação e retorna uma resposta com o recurso solicitado (como uma página HTML, um arquivo JSON etc.) ou uma mensagem de erro, caso algo dê errado.

De acordo com o livro “**HTTP: The Definitive Guide**”, de Brian Totty e David Gourley (2002), o protocolo HTTP possui as seguintes características:

**Comunicação Cliente-Servidor:** como já evidenciado, o HTTP é baseado em um modelo cliente-servidor. O cliente, geralmente um navegador, envia solicitações ao servidor, que responde com os recursos solicitados.

**Conexões Stateless:** cada requisição HTTP é independente, ou seja, o protocolo não guarda informações sobre requisições anteriores. Isso o torna simples e escalável.

**Métodos HTTP:** esse protocolo define métodos como:

- GET: para recuperar dados.
- POST: para enviar dados ao servidor.
- PUT: para atualizar ou criar dados.
- DELETE: para remover dados.
- HEAD: para obter apenas os cabeçalhos da resposta.

**Mensagens HTTP:** as comunicações consistem em mensagens:

- Request (solicitação): enviada pelo cliente ao servidor.
- Response (resposta): retornada pelo servidor ao cliente.

Ambas possuem cabeçalhos e, opcionalmente, um corpo com dados.

**Endereçamento por URIs:** cada recurso na web é identificado por um Uniform Resource Identifier (URI), como uma URL.

**Códigos de Status:** as respostas incluem códigos que indicam o resultado da solicitação como:

- 200: sucesso.
- 404: recurso não encontrado.
- 500: erro no servidor.

**Uso do TCP/IP:** o HTTP depende do protocolo TCP para comunicação confiável.

**Extensibilidade:** é possível adicionar cabeçalhos e funcionalidades ao protocolo sem quebrar a compatibilidade.

Essas características tornam o protocolo HTTP flexível e amplamente utilizado como o principal protocolo para a web moderna.

**O HTTP é um protocolo de comunicação utilizado para transferir dados entre clientes e servidores na web. Ele é baseado em requisições e respostas e utiliza métodos como GET, POST, PUT e DELETE para manipular recursos.**

### Arquitetura REST

REST é um tipo de arquitetura de software desenvolvido para sistemas distribuídos, principalmente para web. Esse conceito foi apresentado em 2000 por Roy Fielding, em sua tese de doutorado chamada "Architectural Styles and the Design of Network-based Software Architectures", na Universidade da Califórnia.

Roy Fielding é um cientista da computação que, junto de outros pesquisadores, ajudou a escrever a especificação do protocolo HTTP. Na tese dele, o REST é descrito como um conjunto de restrições arquiteturais que, quando aplicadas a um sistema, ajudam a criar uma arquitetura mais escalável, eficiente e confiável. A ideia principal é tratar tudo como um recurso, que pode ser representado e manipulado por uma interface padrão, usando os métodos do HTTP, tipo GET, POST, PUT e DELETE.

A arquitetura REST foi pensada por Roy para resolver problemas de complexidade, falta de padronização e dificuldade de escalabilidade na comunicação entre sistemas distribuídos. Antes da REST, as integrações eram frequentemente baseadas em protocolos proprietários ou soluções pesadas, dificultando a interoperabilidade e a evolução dos sistemas. A REST surgiu como uma solução simples e flexível, utilizando o HTTP como base e estabelecendo princípios claros, como identificação de recursos por URIs, interface uniforme e comunicação sem estado, permitindo maior eficiência e escalabilidade na web.

Um exemplo prático de como a REST melhorou a comunicação na web é o caso das APIs de serviços de mapas, como o Google Maps. Antes da REST, integrar serviços de mapas em uma aplicação exigia protocolos complexos, arquivos grandes de dados geográficos ou software específico instalado no cliente. Isso tornava o processo lento e dependente de soluções proprietárias.

Com a REST, o Google Maps disponibiliza uma API onde cada recurso (como um mapa, rota ou ponto de interesse) é acessado por uma URI. Por exemplo, para obter informações sobre a rota entre dois endereços, basta fazer uma requisição HTTP como esta:

```
```bash
GET
https://maps.googleapis.com/maps/api/directions/json?origin=São
o+Paulo&destination=Rio+de+Janeiro
```
```

A resposta virá em formato JSON, padronizada e fácil de manipular, contendo as informações da rota:

```
```bash
{
  "geocoded_waypoints": [
    {
```



```
"geocoder_status": "OK",
"place_id": "ChIJd7zN_thzj5QRFEWG1k9gJ08",
"types": ["locality", "political"]
},
{
  "geocoder_status": "OK",
  "place_id": "ChIJW-T2Wt7_oQAR7zcnxwYQYmU",
  "types": ["locality", "political"]
}
],
"routes": [
  {
    "bounds": {
      "northeast": {
        "lat": -21.2032,
        "lng": -41.7388
      },
      "southwest": {
        "lat": -23.5505,
        "lng": -46.6333
      }
    },
    "legs": [
      {
```

```
"distance": {  
    "text": "429 km",  
    "value": 429000  
},  
  
"duration": {  
    "text": "5 hours 42 mins",  
    "value": 20520  
},  
  
"start_address": "São Paulo, SP, Brazil",  
"end_address": "Rio de Janeiro, RJ, Brazil",  
"start_location": {  
    "lat": -23.5505,  
    "lng": -46.6333  
},  
"end_location": {  
    "lat": -22.9068,  
    "lng": -43.1729  
},  
  
"steps": [  
    {  
        "distance": {  
            "text": "3.4 km",  
            "value": 3400  
        },  
    },  
]
```

```
        "duration": {
            "text": "5 mins",
            "value": 300
        },
        "html_instructions": "Head northwest on Avenida
Paulista",
        "start_location": {
            "lat": -23.5505,
            "lng": -46.6333
        },
        "end_location": {
            "lat": -23.5478,
            "lng": -46.6286
        },
        "travel_mode": "DRIVING"
    },
    // Outros passos omitidos para simplificação
]
}

],
"overview_polyline": {
    "points": "encodedPolylineString"
},
"summary": "BR-116"
```

```
}  
  
  ],  
  
  "status": "OK"  
  
}
```

Esse formato de resposta simplifica a integração, permite que qualquer linguagem ou plataforma com suporte HTTP consuma o serviço e melhora a experiência dos desenvolvedores e dos usuários finais.

### **Como a arquitetura REST melhora a experiência de comunicação na web utilizando o protocolo HTTP?**

A arquitetura REST utiliza o HTTP de forma padronizada e eficiente, estabelecendo princípios como identificação de recursos por URIs, interface uniforme e comunicação sem estado. Isso simplifica a integração entre sistemas, tornando as APIs mais acessíveis, escaláveis e interoperáveis, além de facilitar o consumo de dados em diferentes formatos, como JSON.

### **O que é Flask?**

[Flask](#) é uma biblioteca em Python que facilita o desenvolvimento de APIs REST. Ele fornece o essencial para o desenvolvimento de APIs, permitindo flexibilidade para adicionar bibliotecas conforme a necessidade do projeto.

A biblioteca Flask é **simples e flexível**, pois permite controlar quais funcionalidades serão adicionadas, e possui ampla adesão da **comunidade**, com forte suporte e boa documentação, sendo ideal para iniciantes.

### **Criando o Ambiente**

Antes de começar, vamos ver como instalar o Flask. Tendo o Python instalado localmente, abra o terminal e execute:

```
```bash  
  
pipinstallFlask=2.3.2  
  
```
```

Esse comando baixa e instala o Flask na versão 2.3.2, o qual é uma versão estável e amplamente utilizada.

### Primeira API com Flask

Crie um arquivo chamado `app.py` com o seguinte código:

```
```python
from flask import Flask #Importa o framework Flask

app = Flask(__name__) # Cria a aplicação Flask

@app.route("/") # Define a rota para o caminho "/"
def home():
    return {"message": "Hello world!"} # Retorna uma resposta
    JSON

if __name__ == "__main__":
    app.run(debug=True) # Inicia o servidor com debug ativado
```
```

Explicação do código:

- `Flask`: classe principal para criar a aplicação.
- `@app.route("/")`: cria um endpoint que responde a requisições feitas ao caminho base (`/`).
- `return {"message": "Hello world!"}`: responde com um dicionário Python, que é automaticamente convertido em JSON.

- `app.run(debug=True)`: inicia o servidor local. O modo debug ajuda a identificar erros durante o desenvolvimento.

A seguir executamos o arquivo:

```
```bash  
python app.py  
```
```

Agora, abra o navegador em `http://127.0.0.1:5000/` para visualizar a mensagem.

### Métodos HTTP com Flask

Como já mencionado, o HTTP é o protocolo base para comunicação entre clientes (como navegadores) e servidores. Ele define vários métodos que indicam a intenção de uma requisição. Vamos entender os principais métodos e implementar exemplos simples utilizando Flask:

- **GET**: é usado para recuperar dados do servidor e não altera o estado do servidor. **Exemplo de uso**: busca de uma lista de usuários ou detalhes de um recurso.
- **POST**: é usado para enviar dados ao servidor, geralmente para criar novos recursos. **Exemplo de uso**: adicionar um novo usuário em um banco de dados.
- **PUT**: usado para atualizar ou criar recursos. É idempotente, ou seja, múltiplas requisições com o mesmo dado produzem o mesmo resultado. **Exemplo de uso**: atualizar informações de um usuário existente.
- **DELETE**: é usado para remover recursos no servidor. **Exemplo de uso**: excluir um usuário.
- **HEAD**: é similar ao GET, mas retorna apenas os cabeçalhos da resposta, sem o corpo. **Exemplo de uso**: verificar se um recurso existe sem baixar os dados completos.

## Exemplos de métodos HTTP com Flask

Segue, agora, um exemplo utilizando os métodos HTTP mencionados com a biblioteca Flask.

Crie o script `app2.py` e insira o seguinte código Python:

```
```python

from flask import Flask, request, jsonify

# Cria a aplicação Flask
app = Flask(__name__)

# Dados simulados
users = {"1": {"name": "Alice"}, "2": {"name": "Bob"}}

# Rota GET - Recupera a lista de usuários
@app.route("/users", methods=["GET"])
def get_users():
    """Exemplo de método HTTP GET"""
    return jsonify({"users": users}), 200

# Rota POST - Adiciona um novo usuário
@app.route("/users", methods=["POST"])
def add_user():
    """Exemplo de método HTTP POST"""
    data = request.json
```

```
user_id = str(len(users) + 1) # Gera um novo ID

    users[user_id] = data

    return jsonify({"message": "User created!", "user":
{user_id: data}}), 201

# Rota PUT - Atualiza um usuário existente
@app.route("/users/<user_id>", methods=["PUT"])
def update_user(user_id):
    """Exemplo de método HTTP PUT"""
    if user_id not in users:
        return jsonify({"error": "User not found!"}), 404
    data = request.json
    users[user_id].update(data) # Atualiza os dados
    return jsonify({"message": "User updated!", "user": {user_id:
users[user_id]}}), 200

# Rota DELETE - Remove um usuário
@app.route("/users/<user_id>", methods=["DELETE"])
def delete_user(user_id):
    """Exemplo de método HTTP DELETE"""
    if user_id not in users:
        return jsonify({"error": "User not found!"}), 404
    deleted_user = users.pop(user_id)
```



```
    return jsonify({"message": "User deleted!", "user":
{user_id: deleted_user}}), 200

# Rota HEAD - Verifica se um usuário existe
@app.route("/users/<user_id>", methods=["HEAD"])
def head_user(user_id):
    """Exemplo de método HTTP HEAD"""
    if user_id not in users:
        return "", 404
    return "", 200

# Inicia o servidor Flask
if __name__ == "__main__":
    app.run(debug=True)
...
```

Execute o script no terminal com o comando:

```
```bash
python app3.py
...
```

Agora, podemos testar cada método do script. Primeiro, vamos testar o exemplo do método GET para recuperar a lista de usuários fazendo a seguinte requisição:

```
```bash
```

```
curl -X GET http://127.0.0.1:5000/users  
...
```

Resposta da requisição esperada:

```
```bash  
  
{  
  "users": {  
    "1": {"name": "Alice"},  
    "2": {"name": "Bob"}  
  }  
}  
...
```

Agora, fazemos a requisição no método POST para adicionar um novo usuário:

```
```bash  
  
curl -X POST -H "Content-Type: application/json" \  
-d '{"name": "Carlos"}' http://127.0.0.1:5000/users  
...
```

Resposta da requisição:

```
```bash  
  
{  
  "message": "Usercreated!",  
}
```

```
"user": {  
    "3": {"name": "Carlos"}  
}  
}  
...
```

Requisição no método PUT para atualizar um usuário existente:

```
```bash  
  
curl -X PUT -H "Content-Type: application/json" \  
-d '{"name": "Alice Updated"}' http://127.0.0.1:5000/users/1  
...
```

Retorno esperado da requisição:

```
```bash  
  
{  
  "message": "User updated!",  
  "user": {  
    "1": {"name": "Alice Updated"}  
  }  
}  
...
```

Requisição no método DELETE para excluir um usuário:

```
```bash
```

```
curl -X DELETE http://127.0.0.1:5000/users/2
...
```

Resposta da requisição esperada:

```
```bash
{
  "message": "User deleted!",
  "user": {
    "2": {"name": "Bob"}
  }
}
...
```

Requisição no método HEAD para verificar se um usuário existe:

```
```bash
curl -I http://127.0.0.1:5000/users/1
...
```

Resposta da requisição:

```
```bash
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 0
...
```

### O que cada método do HTTP faz?

O método GET busca informações existentes no servidor, o POST é para criar recursos, o PUT é para atualizar ou criar um recurso (caso ele não exista), o DELETE remove um recurso, e o HEAD verifica se um recurso existe sem carregar os dados.

### Códigos de status com Flask

Nesta parte da aula, teremos contato com diferentes tipos de **códigos de status HTTP** com Flask. Os códigos de status são essenciais para indicar o resultado da solicitação HTTP do cliente para a API. Os tipos mais comuns são:

- **200:** indica que a solicitação foi bem-sucedida. Esse é o status padrão para respostas que retornam dados sem erros.
- **201:** indica que um recurso foi criado com sucesso. Geralmente usado para requisições **POST**.
- **400:** indica que há algo errado com a solicitação enviada pelo cliente, como dados inválidos ou ausentes.
- **401:** indica que o cliente não está autenticado. É usado quando a API exige autenticação.
- **403:** indica que o cliente não tem permissão para acessar o recurso, mesmo estando autenticado.
- **404:** indica que o recurso solicitado não foi encontrado.
- **500:** indica que houve um erro interno no servidor. Isso geralmente é retornado automaticamente quando há uma exceção não tratada.

### Código completo com todos os exemplos

A seguir é fornecido o script com todos os tipos de código de status HTTP com Flask citados. Crie o script chamado **app3.py** e insira o código Python a seguir:

```
```python
from flask import Flask, request, jsonify, Response
```

```
# Inicia o servidor Flask

app = Flask(__name__)

@app.route("/")
def home():
    """ Exemplo para status 200. """
    status_code = 200
    status_message = Response(status=status_code).status
    return jsonify({
        "status": status_message,
        "data": {"message": "Welcome to the API!"}
    }), status_code

@app.route("/create", methods=["POST"])
def create():
    """ Exemplo para status 201. """
    status_code = 201
    status_message = Response(status=status_code).status
    return jsonify({
        "status": status_message,
        "data": {"message": "Resource successfully created!"}
    }), status_code
```

```
@app.route("/validate", methods=["POST"])

def validate():

    """ Exemplo para status 400. """

    data = request.json

    if not data or "name" not in data:

        status_code = 400

        status_message = Response(status=status_code).status

        return jsonify({

            "status": status_message,

            "data": {"error": "Invalid data or 'name' field is

missing!"}

        }), status_code

        # Caso os dados sejam válidos, retorna sucesso

        status_code = 200

        status_message = Response(status=status_code).status

        return jsonify({

            "status": status_message,

            "data": {"message": "Data isvalid!"}

        }), status_code

@app.route("/private")

def private():

    """ Exemplo para status 401. """

    auth = request.headers.get("Authorization")
```

```
ifnotauth:

    # Caso não tenha autenticação, retorna erro 401
    status_code = 401
    status_message = Response(status=status_code).status

    return jsonify({
        "status": status_message,
        "data": {"error": "Authentication required!"}
    }), status_code

# Retorna mensagem de sucesso se autenticado
status_code = 200
status_message = Response(status=status_code).status

return jsonify({
    "status": status_message,
    "data": {"message": "Welcome to the private area!"}
}), status_code

@app.route("/admin")
defadmin():

    """ Exemplo para status 403. """

    user_role = request.headers.get("Role")

    ifuser_role != "admin":

        # Caso não seja administrador, retorna erro 403
        status_code = 403
        status_message = Response(status=status_code).status
```



```
        return jsonify({
            "status": status_message,
            "data": {"error": "Access forbidden! Admins
only."}
        }), status_code

    # Caso seja administrador, retorna sucesso
    status_code = 200
    status_message = Response(status=status_code).status

    return jsonify({
        "status": status_message,
        "data": {"message": "Welcome, Admin!"}
    }), status_code

@app.route("/item/<int:item_id>")
def get_item(item_id):
    """ Exemplo para status 404. """
    items = {1: "Item 1", 2: "Item 2"}

    if item_id not in items:

        # Caso o item não seja encontrado, retorna erro 404
        status_code = 404
        status_message = Response(status=status_code).status

        return jsonify({
            "status": status_message,
            "data": {"error": "Item not found!"}
```

```
   )), status_code

    # Retorna o item se encontrado

status_code = 200

status_message = Response(status=status_code).status

    return jsonify({

        "status": status_message,

        "data": {"item": items[item_id]}

    }), status_code


@app.route("/error")
def error():
    """ Exemplo para status 500. """
    try:

        # Gera um erro proposital (divisão por zero)

        1 / 0

    exceptZeroDivisionError:

        # Captura o erro e retorna um status 500

status_code = 500

status_message = Response(status=status_code).status

    return jsonify({

        "status": status_message,

        "data": {"error": "Internal server error!"}

    }), status_code
```

```
if __name__ == "__main__":  
    # Inicia o servidor Flask com debug ativado  
    app.run(debug=True)  
  
    ...
```

Agora, vamos testar cada exemplo utilizando o cURL. Execute no terminal:

```
```bash  
python app2.py  
...
```

Acesse o endpoint correspondente (<http://127.0.0.1:5000/>).

Veja o teste do exemplo da rota com código 200:

```
```bash  
curl http://127.0.0.1:5000/  
...
```

Resposta da requisição esperada:

```
```bash  
  
{  
  
    "status": "200 OK",  
  
    "data": {
```

```
    "message": "Hello world!"
  }
}
...
```

Teste do exemplo com código 201:

```
```bash
curl -X POST http://127.0.0.1:5000/create
...
```

Output esperado da requisição:

```
```bash
{
  "data": {
    "message": "Resource successfully created!"
  },
  "status": "201 CREATED"
}
....
```

Teste da rota com código 400:

```
```bash

curl -X POST -H "Content-Type: application/json" \
-d '{"invalid_name": "John"}' http://127.0.0.1:5000/validate
```
```

Output esperado da requisição:

```
```bash

{
  "data": {
    "error": "Invalid data or 'name' field is missing!"
  },
  "status": "400 BAD REQUEST"
}
```
```

Exemplo da rota com código 401:

```
```bash

curl http://127.0.0.1:5000/private
```
```

Resposta da requisição esperada:

```
```bash
```

```
{
  "data": {
    "error": "Authentication required!"
  },
  "status": "401 UNAUTHORIZED"
}
...
```

Teste da rota com código 403:

```
```bash
curl -H "Role: user" http://127.0.0.1:5000/admin
...
```

Output esperado:

```
```bash
{
  "data": {
    "error": "Access forbidden! Adminonly."
  },
  "status": "403 FORBIDDEN"
}
...
```

Exemplo com código 404:

```
```bash
```

```
curl http://127.0.0.1:5000/item/999
```

```
...
```

Output esperado da requisição:

```
```bash
{
  "data": {
    "error": "Item not found!"
  },
  "status": "404 NOT FOUND"
}
```

Teste da rota com código 500:

```
```bash
curlhttp://127.0.0.1:5000/error
...
```

Output esperado:

```
```bash
{
  "data": {
    "error": "Internal server error!"
  },
  "status": "500 INTERNAL SERVER ERROR"
```

```
}  
  
...
```

Esses são os códigos de status básicos mais comuns em APIs. Outros códigos também podem ser utilizados para situações específicas, exemplo: 422 Unprocessable Entity, 429 Too Many Requests).

### Validação de Dados na API

A validação de dados em uma API é essencial para garantir a segurança, integridade e confiabilidade do sistema. Ela previne a entrada de dados inválidos ou maliciosos que podem comprometer o funcionamento da aplicação, causar erros inesperados ou até permitir ataques, como **SQLInjection** ou **Cross-Site Scripting (XSS)**. Além disso, a validação assegura que os dados recebidos seguem o formato esperado, reduzindo inconsistências e facilitando o processamento das informações por outras partes do sistema. Implementar validações robustas também melhora a experiência do usuário, pois mensagens de erro claras e consistentes podem guiá-lo a corrigir problemas de entrada, resultando em uma API mais segura e eficiente.

Para o Python existem algumas opções de libs para validação de dados em APIs como a biblioteca [marshmallow](#) e [Pydantic](#). Aqui, utilizaremos a biblioteca Pydantic para construir exemplos de validação na API desenvolvida com Flask.

### Por que usar a biblioteca Pydantic?

O Pydantic é uma biblioteca poderosa para validação de dados e criação de modelos no Python. Ela se destaca por usar tipagem estática do Python para validar e converter dados de forma automática e eficiente. Em APIs, os dados enviados pelo cliente (entrada) precisam ser validados para garantir que estejam no formato esperado antes de serem processados. O Pydantic facilita esse processo com uma sintaxe simples e integração direta com a tipagem moderna do Python. Portanto:

- O Pydantic valida automaticamente os dados recebidos com base nos tipos definidos;
- Converte valores automaticamente para os tipos esperados (ex.: `str` para `int`);



- Gera mensagens de erro claras para entradas inválidas;
- Integra-se facilmente com frameworks como Flask.

Para instalar o Pydantic:

```
```bash

pipinstallpydantic

```
```

A seguir, teremos dois exemplos de uso do Pydantic para APIs em Flask.

### Exemplo 1 - Validação simples com Pydantic

Neste exemplo, usaremos o Pydantic para validar os dados de um usuário enviado em uma requisição **POST**.

```
```python

from flask import Flask, request, jsonify
from pydantic import BaseModel, ValidationError

app = Flask(__name__)

# Modelo de validação para dados do usuário
class UserModel(BaseModel):
    name: str
    age: int

@app.route("/users", methods=["POST"])
defcreate_user():
```

```
"""
POST: Cria um novo usuário com validação de dados.
"""

data = request.json # Obtém os dados enviados na
requisição
try:
    # Valida os dados usando o Pydantic
    user = UserModel(**data)
    return jsonify({"message": "User created
successfully!", "user": user.dict()})
except ValidationError as e:
    # Retorna erros de validação
    return jsonify({"errors": e.errors()}), 400

if __name__ == "__main__":
    app.run(debug=True)
...
```

Explicando o exemplo, a classe `UserModel` define o formato esperado dos dados do usuário com os campos obrigatórios:

- `name`: uma string.
- `age`: um inteiro.

Fazemos uma requisição `POST` com **dados válidos**:

```
```bash

curl -X POST -H "Content-Type: application/json" \
-d '{"name": "Alice", "age": 25}' http://127.0.0.1:5000/users

```
```

Resposta da requisição:

```
```bash

{
  "message": "User created successfully!",
  "user": {
    "name": "Alice",
    "age": 25
  }
}
```
```

Agora vamos checar o comportamento da requisição **POST** realizada **com dados inválidos** (com uma entrada em string para o **age**, quando o correto seria um inteiro):

```
```bash

curl -X POST -H "Content-Type: application/json" \
-d '{"name": "Alice", "age": "invalid"}'
http://127.0.0.1:5000/users

```
```

Resposta da requisição

```
```bash

{
  "errors": [
    {
      "input": "invalid",
      "loc": ["age"],
      "msg": "Input should be a valid integer, unable to
parse string as an integer",
      "type": "int_parsing".
      "url":
"https://errors.pydantic.dev/2.10/v/int_parsing"
    }
  ]
}
```
```

## Exemplo 2 - Validação com dados opcionais e valores padrão

Neste exemplo, ampliaremos o uso do Pydantic para incluir campos opcionais e valores padrão.

```
```python

from flask import Flask, request, jsonify
from pydantic import BaseModel, ValidationError
from typing import Optional
```

```
app = Flask(__name__)

# Modelo de validação para dados do usuário

class UserModel(BaseModel):

    name: str

    age: int

    email: Optional[str] = None # Campo opcional

    is_active: bool = True # Campo com valor padrão

@app.route("/users", methods=["POST"])
def create_user():
    """
    POST: Cria um novo usuário com validação e valores padrão.
    """
    data = request.json
    try:
        user = UserModel(**data)
        return jsonify({"message": "User created successfully!", "user": user.dict()})
    except ValidationError as e:
        return jsonify({"errors": e.errors()}), 400

if __name__ == "__main__":
    app.run(debug=True)
```

```
...
```

A classe `UserModel` nesse exemplo adiciona dois novos campos:

- `email`: opcional (pode estar ausente na requisição).
- `is_active`: sempre será `True` se não for enviado.

Agora, vamos fazer uma requisição **POST com dados opcionais**:

```
```bash
curl -X POST -H "Content-Type: application/json" \
-d '{"name": "Bob", "age": 30}' http://127.0.0.1:5000/users
...`
```

Resposta da requisição:

```
```bash
{
  "message": "User created successfully!",
  "user": {
    "name": "Bob",
    "age": 30,
    "email": null,
    "is_active": true
  }
}
...`
```

Agora, vamos checar o comportamento da aplicação realizando uma requisição **POST com todos os campos**:

```
```bash
curl -X POST -H "Content-Type: application/json" \
-d '{"name": "Charlie", "age": 22, "email":
"charlie@example.com", "is_active": false}' \
http://127.0.0.1:5000/users
```
```

Retorno da requisição:

```
```bash
{
  "message": "User created successfully!",
  "user": {
    "name": "Charlie",
    "age": 22,
    "email": "charlie@example.com",
    "is_active": false
  }
}
```
```

E, por fim, vamos verificar o comportamento da aplicação para uma requisição **POST com dados inválidos**:

```
```bash

curl -X POST -H "Content-Type: application/json" \
-d '{"name": "Charlie", "age": "invalid"}'
http://127.0.0.1:5000/users

```
```

Resposta da requisição:

```
```bash

{
  "errors": [
    {
      "input": "invalid",
      "loc": [
        "age"
      ],
      "msg": "Input should be a valid integer, unable to parse
string as an integer",
      "type": "int_parsing",
      "url": "https://errors.pydantic.dev/2.10/v/int_parsing"
    }
  ]
}

```
```



Em resumo, o Pydantic facilita a validação e conversão de dados automaticamente, assim como o uso de tipos opcionais e valores padrão permitindo maior flexibilidade nas APIs.

O Flask ainda pode integrar o Pydantic diretamente, simplificando a validação de entradas na aplicação.

EXEMPLO

## MERCADO, CASES E TENDÊNCIAS

O [artigo](#) no blog da AWS, mostra como construir e implantar aplicações de inferência de Machine Learning do zero utilizando o Amazon SageMaker. É demonstrado também o processo de treinamento de um modelo, implantação com endpoint de inferência gerenciado e integração desse endpoint em uma aplicação web desenvolvida com Flask. A API REST criada com Flask permite que usuários enviem dados para o modelo e recebam inferências, possibilitando a construção de soluções interativas.

O artigo também aborda a implantação dessa aplicação em serviços gerenciados, como Amazon ECS ou AWS Elastic Beanstalk, oferecendo um guia completo para criar e operacionalizar soluções de ML.

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, vimos conceitos gerais de protocolo HTTP, arquitetura REST para desenvolvimento de APIs, com a utilização da biblioteca em Python Flask para desenvolvimento de APIs do tipo REST, assim como o uso da biblioteca Pydantic para validação de dados em aplicações combinados com Flask.

As práticas apresentadas nessa etapa, são fundamentais para o desenvolvimento de aplicações de software que utilizam Machine Learning como core da solução.

## REFERÊNCIAS

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000. Disponível em: <[https://ics.uci.edu/~fielding/pubs/dissertation/fielding\\_dissertation.pdf](https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf)>. Acesso em: 17 jan. 2025.

GOURLEY, David; TOTTY, Brian; SAYER, Marjorie; REDDY, Sailu; AGGARWAL, Anshu. **HTTP: The Definitive Guide**. 1. ed. Sebastopol: O'Reilly Media, 2002.

PYDANTIC. **Pydantic Documentation**. Disponível em: <<https://docs.pydantic.dev/latest/>>. Acesso em: 17 jan. 2025.

RELAN, Kunal. **Building REST APIs with Flask: Create Python Web Services with MySQL**. New York: Apress, 2019. DOI: 10.1007/978-1-4842-5022-8.

## GLOSSÁRIO

**Cross-Site Scripting (XSS):** um tipo de vulnerabilidade de segurança em aplicações web que permite a execução de scripts maliciosos no navegador de outros usuários. O ataque ocorre quando dados não validados ou não escapados corretamente são inseridos em páginas da web, permitindo que o invasor roube informações, manipule o conteúdo exibido ou execute ações indesejadas.

**SQL Injection:** técnica de ataque que explora falhas na validação de entradas em aplicações que utilizam bancos de dados SQL. Por meio desse ataque, invasores podem injetar comandos SQL maliciosos para manipular ou acessar dados sensíveis armazenados no banco de dados, podendo até mesmo comprometer o sistema inteiro.

**TCP/IP:** conjunto de protocolos de comunicação que forma a base da internet. O modelo TCP/IP define como os dados devem ser embalados, transmitidos e recebidos entre dispositivos em uma rede, garantindo a interconexão de sistemas heterogêneos. O TCP (Transmission Control Protocol) cuida da transmissão confiável, enquanto o IP (Internet Protocol) gerencia o roteamento e endereçamento dos pacotes de dados.

**UniformResourceIdentifier (URI):** identificador que permite localizar ou acessar recursos na internet, como documentos, imagens ou serviços. Ele é composto por um esquema (como HTTP ou FTP), um caminho e, opcionalmente, parâmetros ou fragmentos. Um exemplo comum de URI é uma URL, usada para acessar páginas web.

**Web:** uma das principais aplicações da internet, composta por um conjunto de documentos interligados e acessíveis por meio de navegadores. Esses documentos, conhecidos como páginas web, são escritos em linguagens como HTML, CSS e JavaScript, e são hospedados em servidores web que utilizam o protocolo HTTP ou HTTPS para comunicação.

## **PALAVRAS-CHAVE**

Flask, APIs REST, Python.

EMEND

The background is a dark blue field filled with numerous small, light blue dots. Overlaid on this are several large, wavy, translucent lines in shades of blue and yellow. A vertical line on the left side has a small 'x' at its base. A circle containing the number '7' is positioned in the upper center, with a thin line extending from it towards the wavy lines. In the bottom right corner, there is a faint, light blue hexagonal shape.

POSTECH